



Towards Efficient Compiler Auto-tuning: Leveraging Synergistic Search Spaces

Haolin Pan*

Institute of Software at Chinese
Academy of Sciences
Beijing, China
Hangzhou Institute for Advanced
Study at University of Chinese
Academy of Sciences
Hangzhou, China
University of Chinese Academy of
Sciences
Beijing, China
panhaolin21@mails.ucas.ac.cn

Yuanyu Wei*

Institute of Software at Chinese
Academy of Sciences
Beijing, China
Hangzhou Institute for Advanced
Study at University of Chinese
Academy of Sciences
Hangzhou, China
University of Chinese Academy of
Sciences
Beijing, China
weiyuanyu22@mails.ucas.ac.cn

Mingjie Xing[†]

Institute of Software at Chinese
Academy of Sciences
Beijing, China
mingjie@iscas.ac.cn

YanJun Wu

Institute of Software at Chinese
Academy of Sciences
Beijing, China
University of Chinese Academy of
Sciences
Beijing, China
yanjun@iscas.ac.cn

Chen Zhao

Institute of Software at Chinese
Academy of Sciences
Beijing, China
University of Chinese Academy of
Sciences
Beijing, China
zhaochen@iscas.ac.cn

Abstract

Determining the optimal sequence of compiler optimization passes is challenging due to the extensive and intricate search space. Traditional auto-tuning techniques, such as iterative compilation and machine learning methods, are often limited by high computational costs and difficulties in generalizing to new programs. These approaches can be inefficient and may not fully address the varying optimization needs across different programs. This paper introduces a novel approach that leverages the synergistic relationships between optimization passes to effectively reduce the search space. By focusing on chained synergy pass pairs that jointly optimize a specific target, our method uses K-means clustering to capture common optimization patterns across programs and forms these pairs into coresets. Leveraging a supervised learning model trained on these coresets, we effectively predict the most beneficial coreset for new programs, streamlining the

search for optimal sequences. By integrating various search strategies, our method quickly converges to near-optimal solutions.

Our approach achieves state-of-the-art performance on ten benchmark datasets, including MiBench, CBench, NPB, and CHStone, demonstrating an average reduction of 7.5% in Intermediate Representation (IR) instruction count compared to Oz. Furthermore, this set of chained synergy pass pairs is also well-suited for iterative search studies by other researchers, as it enables achieving an average codesize reduction of 13.9% compared to Oz with a simple search strategy that takes only about 5 seconds, outperforming existing search-based techniques in the initial pass search space across five datasets.

CCS Concepts: • Software and its engineering → Compilers; Search-based software engineering; • Computing methodologies → Machine learning.

Keywords: Compiler, Auto-tuning, Optimization

ACM Reference Format:

Haolin Pan, Yuanyu Wei, Mingjie Xing, YanJun Wu, and Chen Zhao. 2025. Towards Efficient Compiler Auto-tuning: Leveraging Synergistic Search Spaces. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization (CGO '25)*, March 01–05, 2025, Las Vegas, NV, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3696443.3708961>

*Both authors contributed equally to this research.

[†]Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

CGO '25, March 01–05, 2025, Las Vegas, NV, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1275-3/25/03

<https://doi.org/10.1145/3696443.3708961>

1 Introduction

Compilers offer a range of optimization passes to enhance program performance by reducing the consumption of resources such as memory, CPU, and storage. Determining the optimal sequence of passes manually is challenging, especially as the number of passes increases. For instance, with 124 passes, the number of possible sequences of length 45 exceeds the number of atoms in the universe [22]. To address this challenge, compiler auto-tuning techniques automate the process of finding the optimal sequence, significantly reducing the need for manual optimization.

Considerable progress has been made in improving auto-tuning techniques. Iterative compilation methods were effective in the earlier stages of compiler optimization, while recent machine learning techniques have been leveraged to guide the search process [7]. These include performance modeling to accelerate iterations, direct machine learning approaches to predict pass sequences, and reinforcement learning techniques to find optimal pass sequences [4, 7, 21, 27]. Despite the advancements, existing methods have limitations. For example, search-based tools like Opentuner [3], BOCA [9], CompTuner [28], and TPE [6] focus on finding optimal sequences for individual programs, which can be time-consuming. On the other hand, Autophase [18] uses reinforcement learning to generalize across programs by learning different program representations, allowing it to reason about promising sequences for unseen programs. However, this approach relies on well-designed program representations and model structures, and may not perform as well for unseen programs compared to direct search methods.

In this paper, we introduce a novel approach for finding the optimal pass sequence by leveraging the interactions between optimization passes to optimize a specific target. Our method focuses on identifying chained synergy pass pairs that work together to enhance program optimization. We first define a formula to identify these pairs for each program in the training set, capturing their cooperative effects. To generalize these pairs across different programs, we cluster the chained synergy pass pairs using K-means, forming coresets that represent common optimization strategies. Building on this foundation, we employ a supervised learning model to intelligently select the most suitable chained synergy pass pair set from these coresets for unseen programs in the test set. By narrowing the search space to these selected sets, our method effectively guides the search strategy to derive near-optimal pass sequences. Our approach, which integrates prediction and search, achieves state-of-the-art performance across ten datasets with an average reduction of 7.5% in Intermediate Representation (IR) instruction count compared to Oz. Furthermore, by constructing a search space using all chained synergy pass pairs from five training sets, aimed at reducing codesize, we successfully guided the reduction of codesize across the corresponding test sets of

these datasets. By applying a genetic algorithm that takes only 6 seconds to this search space, our approach achieves state-of-the-art codesize reduction results, with an average reduction of 13.9%, outperforming other search-based methods applied within the 124-pass search space, such as BOCA, OpenTuner, and TPE.

In summary, our contributions are as follows:

- We propose a novel method to identify chained synergy pass pairs that enhance compiler optimization. The method evaluates the impact of applying combinations of passes on a specific target and identifies pairs that provide significant additional optimization benefits. This approach facilitates the discovery of potential optimization opportunities and reduces redundant optimization operations.
- We cluster the chained synergy pass pairs identified by our algorithm for each program. This clustering process creates coresets of chained synergy pass pair sets that capture common optimization strategies, enabling effective reuse of optimization knowledge across different programs.
- Our approach demonstrates state-of-the-art performance in reducing IR instruction count and codesize, outperforming existing techniques while generalizing well to unseen programs with minimal time overhead.
- We provide a set of search strategies for the chained synergy pass pair set, including random walk, greedy algorithm, genetic algorithm, and reinforcement learning, with the genetic algorithm (GA) showing the best performance.

2 Motivation

2.1 Exploring Valuable Search Spaces

When automating the search for an optimal pass sequence for a program, a significant challenge arises due to the enormous and redundant search space. To address this, if we can eliminate passes that are not beneficial for the program and reduce the search space, the complexity of the search can be significantly reduced. Therefore, we attempt to select passes that are beneficial (i.e., those that reduce the IR instruction count) to construct a more focused search space and conduct a random search experiment for pass sequences. In our experiment, we set the length of the randomly searched pass sequences to 20. Meanwhile, the complete search space consists of 124 transformation passes from LLVM 10.0.0 *opt*. To minimize data variability, we perform 1,000 searches in the complete search space, whereas the reduced beneficial search space only requires 100 searches. The objective of these experiments is to optimize the percentage reduction in IR instruction count.

From Table 1, it can be observed that for programs in the Cbench suite (where only programs with fewer than 10K instructions are retained), the average IR instruction

count reduction percentage of sequences in the beneficial search space is close to the maximum reduction percentage. Meanwhile, in the complete search space, there is a significant gap between the two, indicating that more valuable pass sequences can be found in the beneficial search space. Although the maximum IR instruction count reduction percentage of random sequences in the complete search space is close to that in the beneficial search space, it comes at the cost of ten times the search effort. If we can identify a more valuable search space for programs, the difficulty of finding an excellent pass sequence can be greatly reduced.

2.2 Leveraging Synergistic Relationships Between Passes

Utilizing the relationships between passes to search for pass sequences has been proven to be effective [26]. To observe this relationship, we conducted an experiment considering only the impact between two consecutive passes. In this experiment, the program was instructed to choose the pass at each step that maximized the efficiency of the previous pass. Specifically, at each step within the complete search space, the program selects the pass that results in the greatest reduction in IR instruction count under the current state, continuing until the sequence reaches a length of 20.

As shown in the **Greedy** column of Table 1, the final selected sequence achieves a reasonably high average IR instruction count reduction percentage. This finding suggests that considering only the relationship between consecutive passes can yield near-optimal results. However, this approach is time-consuming, requiring nearly 2,500 searches per program. Additionally, we observed that the selection process ultimately converged to passes that were ineffective in further optimizing the target. The reason for this behavior

Table 1. IR instruction count reduction percentages compared to O_0 for Cbench programs across different search methods and search spaces.

Filename	Beneficial		Complete		Greedy
	Avg	Max	Avg	Max	
adpcm	56.2%	63.3%	17.2%	61.0%	61.6%
rijndael	41.1%	47.0%	16.1%	46.0%	47.0%
stringsearch2	52.9%	54.6%	10.8%	52.7%	54.6%
dijkstra	40.6%	42.7%	8.7%	40.9%	42.7%
qsort	52.2%	57.5%	8.9%	54.4%	57.5%
crc32	52.3%	52.9%	11.6%	52.5%	52.9%
bitcount	48.7%	53.0%	16.8%	51.0%	53.4%
blowfish	47.4%	50.4%	16.6%	48.1%	51.9%
patricia	59.8%	69.3%	14.7%	68.0%	68.7%
stringsearch	50.1%	54.9%	15.6%	53.9%	54.9%
sha	52.0%	55.7%	12.9%	55.3%	55.2%
Average	50.3%	54.7%	13.6%	53.1%	54.6%

is the inability to find passes that can further reduce the current program's IR instruction count, which is a drawback of greedy algorithms that are prone to leading to local optima.

By narrowing the search space to include only beneficial passes and leveraging synergistic relationships between passes, we can significantly improve the efficiency of finding the optimal pass sequence. Simple experiments demonstrate that this approach achieves more effective optimization with fewer search attempts. These findings inspire us to refine our method, aiming to enhance the search process while reducing computational effort.

3 Related Work

Optimizing compiled programs has been a key area of research, aimed at enhancing performance, reducing codesize, and improving energy efficiency. In the initial phases of research in this domain, a range of optimization techniques were explored, such as stochastic search and heuristic algorithms. J. Bergstra et al. explored the effectiveness of stochastic search and introduced a novel model-based sequential optimization algorithm, TPE [6], for tuning deep belief networks. Y. Chen et al. proposed RIO [10], which involves randomly assigning the enable/disable status of each optimization flag in each iteration. Ansel et al. presented OpenTuner [3], a framework for constructing domain-specific multi-objective program auto-tuners. This framework introduces the concept of employing a collection of search techniques in auto-tuning, which enables the collaborative application of multiple techniques to identify an optimal solution and facilitates a more robust search than using a single technique. U. Garciarena utilized a Genetic Algorithm (GA) [16] to optimize compiler flag selection. An initial population of random flag configurations is iteratively refined through crossover and mutation operations, with each generation evaluated for program performance and the fittest configurations selected for further propagation. J. Chen and colleagues proposed BOCA [9], a new method for identifying the most influential optimization flags. This method employs a random forest model to identify influential optimization flags, which are then used for a comprehensive search, thus improving search efficiency. M. Zhu et al. proposed CompTuner [28], a multiple-phase learning-based compiler auto-tuning technique that reduces time costs and enhances performance. Unlike BOCA, CompTuner builds a prediction model using a diverse dataset and optimizes the search for effective sequences with an improved particle swarm optimization algorithm.

The studies presented above focused on the search for a single program, which is a relatively time-consuming process. On the other hand, to reduce the time spent on searching, researchers have initiated investigations into the direct prediction of near-optimal sequences for a given program through the utilization of machine learning models. Q. Huang et al. proposed Autophase [18], a framework that utilizes deep

reinforcement learning to identify the optimal pass sequence for program optimization. Although machine learning methods can significantly reduce search time, their performance is often suboptimal when applied to programs that have not been previously encountered.

Three additional studies are pertinent to our work. One study by Y. Liang et al. proposed a method for reducing IR instruction count based on coreset and normalized value prediction. This method identifies the optimal sequence of compiled passes within 45 compiled calls through a graph neural network learning strategy [22]. Despite its efficiency, the use of a fixed set of sequences in the coreset imposes certain limitations on its generalization ability across different programs. Another study conducted by S. Park proposed SRTuner [26], a tool that enhances program performance by analyzing and exploiting synergistic relationships between passes. It employs multi-stage structures and multi-armed bandit algorithms to efficiently explore the optimization space. However, SRTuner requires identifying synergistic relationships for each individual program, which is a time-consuming process and lacks generalizability. Additionally, a study on synergistic relationships between passes, designated as ICMC [23], also proposes the concept of synergism. If for a given program P , the pass sequence (A, B) outperforms both pass A and pass B individually, as well as the sequence (B, A) , it indicates that pass A enhances the effectiveness of pass B , implying that pass B depends on pass A . However, this approach to identifying synergistic pairs is more restrictive and may result in the exclusion of some potentially useful pairs. Furthermore, subsequent pruning operations conducted in this study to identify the optimal sequence on the constructed synergy graphs may lead to the final outcome being trapped in a local optimum.

We propose a method for constructing an optimization search space based on synergy. First, we define a chained synergy pass pair that is suitable for connecting. Each search space consists of multiple chained synergy pass pairs and represents the optimization opportunities for a specific class of programs. By leveraging multiple search spaces, the method can effectively generalize to accommodate various program types. Ultimately, a program can select and connect pass pairs from the search space that align with its optimization needs, thereby identifying an appropriate optimization sequence to improve performance.

4 Approach

The principal objective of our methodology is to facilitate the rapid identification of near-optimal pass sequences for previously unseen programs. The proposed method is illustrated in Figure 1. Initially, the dataset is divided into three subsets: the training set, the validation set, and the test set. In the first step, the chained synergy pass pairs for each program in the training set are identified. These pairs represent

combinations of compiler passes that work together in a way that enhances the performance of compilation optimization while also helping to control the search space [20]. These sets are then clustered and merged to form k coresets of chained synergy pass pairs. Next, the scores of these coresets for each program in the training set are computed, resulting in a score vector that represents the program's probability distribution across different coresets. After training the neural network, the model predicts the probability distributions for the programs in the test set and selects the set with the highest predicted probability for further exploration. Finally, a search method is employed to find the near-optimal sequence within the selected set. To provide a more intuitive demonstration of the effectiveness of our methodology, we choose IR instruction reduction as the optimization target.

4.1 Finding Chained Synergy Pass Pairs

Algorithm 1 Finding Chained Synergy Pass Pairs for Program

```

1: Input: Program  $P$ , set of compiler transformation passes  $O$ 
2: Output: Set of chained synergy pass pairs  $S$ 
3:  $S \leftarrow \emptyset$ 
4: Compute  $V_P$ , the IR instruction count of the  $P$ 
5: for each optimization pass  $B \in O$  do
6:   Apply pass  $B$  to  $P$  and obtain the IR instruction count  $V_{P_B}$ 
7:   if  $V_{P_B} < V_P$  then
8:     for each optimization pass  $A \in O$  do
9:       Apply pass  $A$  followed by pass  $B$  to  $P$  and obtain the IR instruction count  $V_{P_{AB}}$  compared to  $O_0$ 
10:      if  $V_{P_{AB}} < V_{P_B}$  then
11:         $S \leftarrow S \cup \{(A, B)\}$ 
12:      end if
13:    end for
14:  end if
15: end for
16: Return  $S$ 

```

We apply Algorithm 1 to identify chained synergy pass pairs for each of the M (19,603) programs in the training set, using LLVM 10.0.0 and the 124 transformation passes available in the *opt* tool. This process results in M' (8,257) sets of chained synergy pass pairs, where $M' < M$ because some programs do not yield any chained synergy pass pairs. The algorithm first evaluates the impact of each pass individually on the program's IR instruction count. It then examines all possible pairs of passes to determine if their combined application results in a greater reduction in instruction count compared to applying the second pass alone. Specifically, a chained synergy pass pair is defined as one where the application of the first pass followed by the second pass

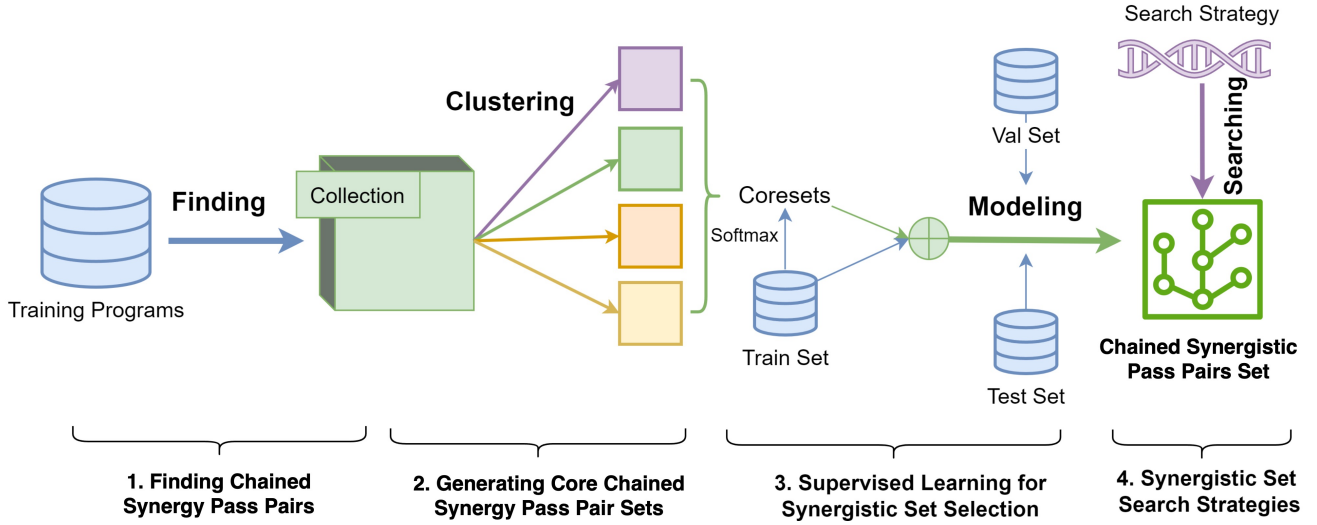


Figure 1. Overview of our approach.

achieves a more significant optimization effect than applying the second pass alone. This approach helps identify pass combinations that, when connected, can further optimize the program, enabling more effective overall optimization.

Through observation, we categorize these chained synergy pass pairs into two types: complementary and contributing. For example, the passes `--dce` (Dead Code Elimination) and `--mem2reg` (Promote Memory to Register), as shown in Figure 2a, act on different parts of the code independently. Their combined effect is the same regardless of the order of application, as they complement each other without overlap. In contrast, consider `--simplifycfg` (Simplify Control Flow Graph) and `--instcombine` (Instruction Combining), as illustrated in Figure 2b. The `--simplifycfg` pass optimizes the control flow graph by removing unnecessary branches and merging basic blocks, which improves the program's flow. However, its optimization potential can be limited by the presence of redundant instructions. When `--instcombine` is applied first, it eliminates these redundant instructions, allowing `--simplifycfg` to perform more effective optimizations on the control flow, resulting in a more optimal outcome. Our chained synergy pairs encompass both types of situations described above.

4.2 Generating Core Chained Synergy Pass Pair Sets

We merge sets of chained synergy pass pairs for similar programs based on the following observations: (1) From Figure 3, it is evident that the effectiveness of different chained synergy pairs varies. Some of the E (1,548) enumerated chained synergy pairs are effective across multiple programs, while others are effective for only a few. Merging sets of chained synergy pairs for similar programs can significantly reduce the number of highly similar sets. (2) In our search for chained synergy pass pairs across different

programs, we observed significant variability in the number of such pairs. Some programs have hundreds of chained synergy pairs, while others have only a few. This variation does not necessarily indicate a lack of optimization potential. Our strict definition aims to identify high-value pairs, and some programs might require specific passes to uncover additional chained synergy pairs. By merging these sets, we can explore optimization opportunities more broadly across similar programs. As programs undergo changes during the search process with passes being applied, their characteristics may shift, potentially revealing new chained synergy pairs. Thus, similar programs, representing variations in features, contribute to a more comprehensive search space that may include novel chained synergy pairs.

Algorithm 2 Constructing Reward Vector for Program

- 1: **Input:** Program P , set of E chained synergy pass pairs $\mathcal{P}$, where $\mathcal{P}$ is the set of chained synergy pass pairs enumerated from all M' programs
 - 2: **Output:** Vector v of length E
 - 3: $v \leftarrow [0]^E$
 - 4: **for** $j = 1$ to E **do**
 - 5: Let (A_j, B_j) be the j -th chained synergy pass pair in $\mathcal{P}$
 - 6: **if** (A_j, B_j) is a chained synergy pass pair for P **then**
 - 7: Apply pass pair (A_j, B_j) to P and compute the IR instruction count reduction percentage r_j
 - 8: $v[j] \leftarrow r_j$
 - 9: **else**
 - 10: $v[j] \leftarrow 0$
 - 11: **end if**
 - 12: **end for**
 - 13: **Return** v
-

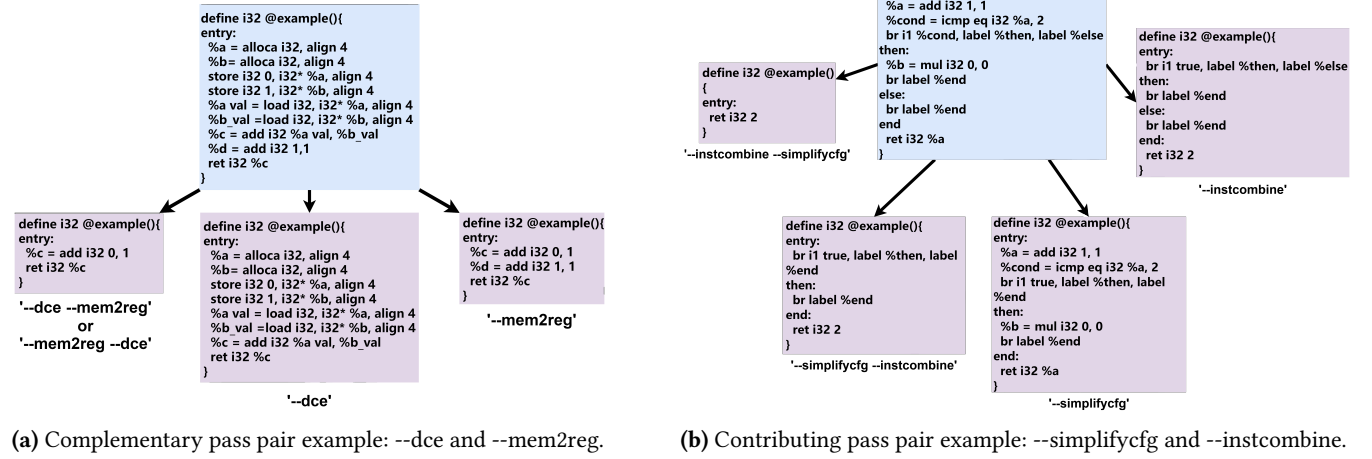


Figure 2. Examples of complementary and contributing pass pairs in LLVM optimization.

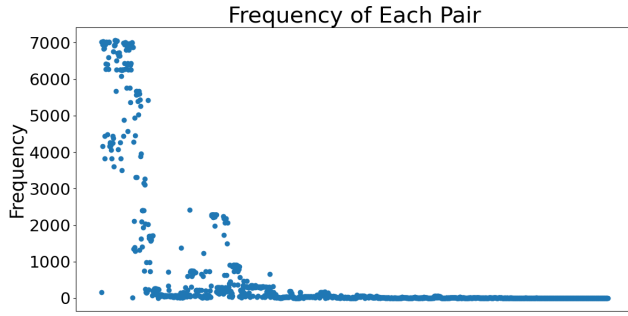


Figure 3. Statistics on the total number of occurrences of each chained synergy pass pair across the sets of chained synergy pass pairs for different training programs.

We use Algorithm 2 to construct a reward vector representation for each program based on the set of E chained synergy pass pairs enumerated from the non-empty M' sets. For each program, a vector of length E is created, where each entry corresponds to a specific chained synergy pass pair drawn from these sets. The algorithm checks whether each pass pair exists in the program's own chained synergy pass pair set. If a pass pair is present, its impact on the program's IR instruction count reduction is calculated and recorded in the vector. If the pass pair is not part of the program's set, the corresponding entry in the vector is set to zero. Through this approach, the generated reward vector effectively reflects the optimization effects of different chained synergy pass pairs on the program. This further highlights the program's intrinsic characteristics, aiding in the subsequent identification of programs with similar features.

Once the reward vectors for M' programs are constructed, we proceed to apply the K-means clustering algorithm to group similar programs. K-means is a widely used clustering algorithm that partitions m data points (in this case, reward

vectors v) into k clusters, such that each data point belongs to the cluster with the nearest mean [2]. The clustering process aims to minimize the within-cluster sum of squares (WCSS), which is defined as:

$$WCSS = \sum_{j=1}^k \sum_{v \in C_j} \|v - \mu_j\|^2, \quad (1)$$

where C_j is the set of points in cluster j , μ_j is the mean of cluster j , and $\|\cdot\|$ denotes the Euclidean distance.

By taking the union of chained synergy pass pair sets for programs within the same cluster, we construct k distinct chained synergy pass pair coresets. These coresets capture the common optimization potential across similar programs while minimizing redundancy. By providing comprehensive coverage of optimization opportunities, each coreset enables programs to select the most relevant pass sequences according to their specific needs.

4.3 Supervised Learning for Synergistic Set Selection

After obtaining the k core chained synergy pass pair sets, we employ a supervised learning approach to select the most suitable chained synergy pass pair set for unseen programs. From each chained synergy pass pair set, we randomly select 100 pass sequences with lengths ranging from 4 to 20. Sequences that are too long may introduce repetitive subsequences, while sequences that are too short might not achieve sufficient optimization. For each program i in the training set, we construct a vector of length k , where each entry j represents the maximum reduction in IR instruction count achieved by applying the 100 pass sequences from the j -th core chained synergy pass pair set. This vector effectively captures the preference of program i for the k chained synergy pass pair sets. Specifically, this preference is defined

by:

$$p_{ij} = \frac{100}{\max_{n=1} \frac{InstC_i(O_0) - InstC_i(s_{jn})}{InstC_i(O_0)}} \quad (2)$$

where s_{jn} denotes the n -th sequence in the j -th chained synergy pass pair set, $InstC_p(s_{jn})$ represents the IR instruction count of program i under the pass sequence s_{jn} , and p_{ij} represents the preference of program i for the j -th chained synergy pass pair set. Here, O_0 denotes the original program, i.e., the program without any optimization applied.

We use an improved version of ProGraML [12] as program features and a Graph Edge Attention Network (GEAN) as the prediction model to learn the relationship between feature vectors and score vectors. GEAN enhances the standard Graph Attention Network (GAT) by incorporating dynamic edge representations [8], which are updated at each layer, and by preserving directional information in graph edges. These improvements enhance the model's generalization capability [22]. We employ the Kullback–Leibler (KL) divergence to supervise the training of the GEAN network. For each program i , we compute its score vector $D_i = [p_{i0}, p_{i1}, \dots, p_{ik}]$ across all k chained synergy pair sets. During training, we use KL divergence-based cross-entropy as the loss function. Given a feature vector o_i for program i , the model outputs a probability distribution $a_i = f(o_i)$. The loss function is defined as follows:

$$L(D_i, a_i) = - \sum_{j=1}^k \text{Softmax}(D_i/T)_j \log(a_{ij}) \quad (3)$$

where T is a temperature parameter.

Our supervised learning approach trains a GEAN graph neural network to predict the probability distribution of the k core chained synergy pass pair sets, learning the relationship between program features and their preferences. This method requires only a single training session and eliminates the need for future retraining. Once trained, the model identifies the optimal chained synergy pass pair set for unseen programs by selecting the set with the highest probability. We then explore strategies for optimizing programs further within these sets.

4.4 Synergistic Set Search Strategies

For each program, we search for pass sequences within the chained synergy pass pair set selected by the model, which is similar to but much larger than the program's own set of chained synergy pairs. It is feasible to explore within this set because, even though each pass application modifies the program state, previously effective chained synergy pairs may still be applicable and beneficial. This is because the program, despite its changes, may continue to exhibit similar optimization preferences. Figure 4 illustrates some favorable scenarios for connecting two chained synergy pass pairs. In Figure 4a, both (A, B) and (B, C) are complementary pass

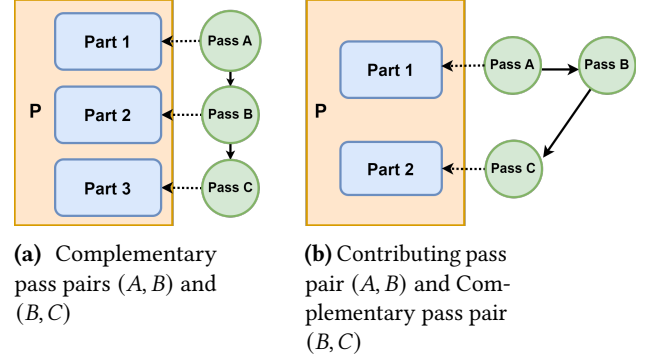


Figure 4. Favorable Situations for Chained Synergy Pass Pair Connections

pairs, and passes A and C also have a complementary relationship. Consequently, the sequence formed by combining (A, B) and (B, C) will optimize all three parts of the program simultaneously. In Figure 4b, (A, B) is a contributing pass pair, and (B, C) is a complementary pass pair. Clearly, *part1* optimized jointly by (A, B) will not affect *part2* optimized by pass C . The newly formed sequence will achieve the maximum benefit by connecting these pass pairs. Difficult-to-analyze connection scenarios can be viewed as exploration strategies to avoid getting trapped in local optima. As mentioned in Section 4.2, clustering merges the sets of chained synergy pass pairs from similar programs, ensuring that the merged set includes more pass pairs that may be advantageous for connection.

We explored several search strategies to find the most effective pass sequence within these sets. These strategies range from simple random exploration to more sophisticated approaches that leverage domain knowledge and machine learning techniques. Specifically, we represent each chained synergy pair set as a directed graph, where nodes represent passes and directed edges represent chained synergy pass pairs. We then use the following strategies to explore this graph:

Random Strategy: The random strategy serves as the simplest method for selecting optimal pass sequences by randomly generating 100 different pass sequences within a synergistic set. For each program, we randomly generate 100 pass sequences, with each sequence constrained to a maximum length of 20 and a minimum length of 4. The sequence that yields the minimum IR instruction count among these 100 randomly generated sequences is selected as the final result.

Greedy Strategy: The greedy strategy for optimizing pass sequences works by iteratively selecting passes that minimize the IR instruction count. Initially, we calculate the IR instruction count for each pair of passes applied from the synergistic set. The pass pair with the smallest IR instruction count is chosen as the starting point. From this

initial pair, the algorithm continues to expand the sequence by adding passes that follow the last pass in the current sequence, aiming to further reduce the IR instruction count. In each iteration, a candidate pass is added to the current sequence to generate potential pass sequences. The pass that results in the lowest IR instruction count is then appended to the sequence. This process is repeated until the sequence reaches a predefined maximum length of 20 passes. We keep track of the intermediate results generated at each iteration, and the final output of the greedy algorithm is the sequence that achieves the minimum IR instruction count across all iterations.

Genetic Algorithm: The genetic algorithm (GA) strategy involves evolving a population of candidate solutions through iterative processes of selection, crossover, and mutation to identify an optimal pass sequence that minimizes the IR instruction count. The key steps in our GA approach include generating an initial population, selecting individuals based on fitness scores, performing crossover to combine promising traits, and applying mutations to introduce diversity. Each step is designed to balance exploration and exploitation, ensuring convergence toward high-quality solutions.

- **Initial Population Generation:** The initial population is generated using the results from the random strategy. This approach provides a diverse set of starting points that serve as potential solutions for the evolutionary process.
- **Selection and Crossover:** The selection step identifies the most suitable individuals for reproduction based on their fitness scores, which are calculated as the percentage reduction in IR instruction counts compared to the original program. Individuals are ranked, and a selection rate is applied to choose parents. If two individuals share at least one common gene point, they are selected for crossover; otherwise, the two highest-scoring individuals are chosen. As illustrated in Figure 5a, the crossover process involves selecting a common gene point (common node in the Figure) and swapping segments of the two parents to create offspring. For two sequences $P_iP_qP_kP_m$ and $P_iP_jP_kP_n$, the common node is P_k . During the crossover, the paths following P_k in both sequences are swapped, resulting in the offspring sequences $P_iP_qP_kP_n$ and $P_iP_jP_kP_m$.
- **Mutation:** The mutation process, depicted in Figure 5b, introduces additional diversity into the population by modifying existing solutions. A mutation operation is triggered at a specified mutation rate, where a mutation point is chosen—typically a node with multiple successor nodes. From this point, a random outgoing edge is selected, and unvisited subsequent nodes are added to form a new mutated path. This operation continues until no further new nodes can be found.

For example, as shown in Figure 5b, in the sequence $P_iP_qP_kP_m$, nodes P_i , P_q , P_k each have an out-degree greater than 1 and are considered potential mutation points. If P_q is randomly selected as the mutation point, the sequence is mutated to $P_iP_qP_jP_k$. The resulting mutated path is then added back into the population as a new candidate solution.

Reinforcement Learning: The reinforcement learning (RL) strategy utilizes an agent-based approach to search for optimal pass sequences within a synergistic set S predicted by a supervised model for a given program P . The key idea is to employ an action masking mechanism to dynamically constrain the search space, guiding the agent toward constructing valid and effective sequences. The agent starts by randomly choosing a chained synergy pair (A, B) from the set S . After selecting this pair, the search space is dynamically updated to form a new subset S_{step1} , which contains all chained synergy pairs beginning with B . All pairs not in S_{step1} are masked to restrict future choices. The agent then selects the next chained synergy pair from this updated set. For example, if the agent chooses (B, D) , the search space is updated again to form S_{step2} , containing only pairs starting with D . This process continues iteratively, constructing the pass sequence step-by-step until a maximum number of steps is reached or no further valid actions are available. We use the 56 features in Autophase to represent the state space, capturing the state of the IR. The reward at each step is the reduction ratio of the IR instruction count compared to the previous step. The total reward is the cumulative reduction ratio across all steps, measuring the effectiveness of the pass sequence.

These search strategies—random, greedy, genetic algorithm and reinforcement learning—offer different approaches to finding efficient pass sequences from a synergistic set. Simple methods like random and greedy provide straightforward solutions, while the genetic algorithm and reinforcement learning use more sophisticated techniques to explore the search space. By employing these search strategies in conjunction with our chained synergy pass pair set, we aim to optimize programs and reduce the number of IR instructions.

5 Experiment

5.1 Experimental Setup

As shown in Table 2, we divide the ten datasets into training, validation, and test sets. These datasets are sourced from CompilerGym [14]. Before splitting, we filtered the original programs to retain only those with fewer than 10K IR instructions. The 10K instructions threshold is based on GEAN-NVP [22], influenced by GPU memory limits and the need for fine-grained optimizations, as larger programs may respond differently to optimization sequences. We aimed for consistency with the training, validation, and test sets in

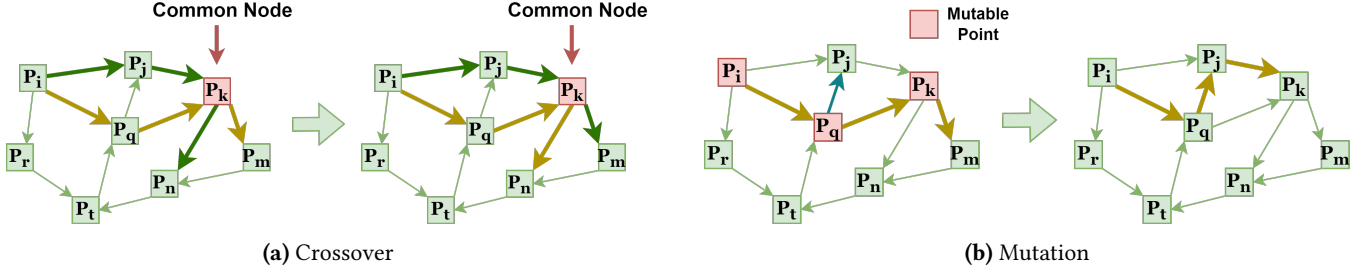


Figure 5. (a) Crossover at the intersection of the two parental paths and exchange of sequences behind the crossover point. (b) When there is a gene point in an individual with an out-degree greater than 1, a mutation is performed at that point with a certain probability.

Table 2. CompilerGym dataset types and training splits. The hand-curated datasets are employed exclusively for the purpose of evaluating generalization to real-world program domains at test time.

Type	Dataset	Train	Val	Test
Uncurated	blas-v0	133	28	29
	github-v0	7,000	1,000	1,000
	linux-v0	4,906	1,000	1,000
	opencv-v0 [11]	149	32	32
	poj104-v1 [24]	7,000	1,000	1,000
	tensorflow-v0 [1]	415	89	90
Curated	cbench-v1 [15]	0	0	11
	mibench-v1 [17]	0	0	40
	chstone-v0 [19]	0	0	12
	npb-v0 [5]	0	0	121
Total	-	19,603	3,149	3,335

GEAN-NVP, removing some datasets due to accessibility or size.

The effectiveness of each dataset is measured by the percentage reduction in both IR instruction count or codesize compared to Oz. The codesize is determined using the *llvm-size* tool, which measures the size of the *.text* section of the program, distinct from the IR instruction count. To evaluate performance, we use the following formula:

$$\bar{I}_{Oz} = \text{MeanOverOz} := \frac{1}{|P|} \sum_{p \in P} \frac{I_{Oz}^p - I_{\pi_\theta}^p}{I_{Oz}} \times 100\%$$

Where P represents the dataset, and I indicates the reduction effect. Specifically, I_{Oz}^p is the code's size after applying the Oz optimization (whether measured in instructions or codesize), and $I_{\pi_\theta}^p$ is the size after applying the optimization strategy. This metric reflects the percentage reduction in IR instructions and codesize, providing a consistent measure of optimization effectiveness across different datasets.

Table 3. Comparison of the average percentage reduction in IR instruction count achieved by our method versus other methods, using *opt Oz* as the baseline, averaged across the test sets of ten datasets.

Method	$\bar{I}_{Oz}$	Samples/Program
GEAN-NVP [22]	3.7%	45
GEAN-NVP (Oracle) [22]	4.8%	2,250
Random (Our)	6.7%	100
Greedy (Our)	4.9%	1,118
GA (Our)	7.5%	1,000
Autophase-RL-PPO (Our)	6.5%	10,000

5.2 Main Experiment

5.2.1 Prediction-based experiments. Table 3 presents the percentage reduction in IR instruction count across all datasets for various methods, relative to the reduction achieved by Oz. GEAN-NVP is one of the most effective methods for reducing IR instruction counts across a broad range of programs and serves as the primary baseline for our comparisons. GEAN-NVP is capable of directly predicting a pass sequence of length 45 through the model, but for each pass applied, it samples 45 times and selects the subsequence with the best result. In comparison, our random strategy, with just 55 additional samples, achieves nearly double the reduction in IR instruction count. When compared to GEAN-NVP (Oracle), our method uses only 1/22 of its sampling budget while achieving 1.4 times the reduction in IR instruction count. Moreover, our method consistently outperforms GEAN-NVP (Oracle) across different search strategies within the chained synergy pass pair search space, highlighting the effectiveness of this search space. Additionally, the pass sequences generated by our method have a length of 20, which is shorter than the sequence lengths of 45 for GEAN-NVP and 97 for Oz.

Table 4 further illustrates the performance of four search strategies in the chained synergy pass pair search space

Table 4. Performance of our different search strategies on individual datasets’ test sets, showing the percentage reduction in IR instruction count relative to *opt* Oz.

Dataset	RL	Random	Greedy	GA
blas-v0	2.8%	2.9%	1.4%	4.6%
cbench-v1	4.8%	4.8%	4.5%	5.8%
chstone-v0	10.5%	10.2%	10.6%	11.9%
mibench-v1	3.9%	3.1%	2.2%	3.7%
npb-v0	20.6%	24.4%	11.7%	25.9%
opencv-v0	5.7%	5.4%	4.6%	5.9%
tensorflow-v0	6.2%	5.8%	5.8%	6.1%
github-v0	1.0%	1.0%	1.0%	1.1%
linux-v0	0.6%	0.6%	0.6%	0.6%
poj104-v1	8.5%	8.6%	6.6%	9.6%
Average	6.5%	6.7%	4.9%	7.5%

across different datasets. It is evident that GA generally exhibits the best performance across most datasets. We analyze the effectiveness of these four search strategies in conjunction with the average number of samples. **1) Random strategy and GA show the best overall performance.** Finding a valuable pass sequence in the chained synergy pass pair search space is relatively straightforward; simply sampling 100 sequences randomly can yield high-quality pass sequences. GA, as an optimization of the random strategy, selects high-quality pass sequences in each generation, which are then crossed over to generate even better sequences. Mutation also introduces exploration features to these high-quality sequences. **2) The greedy algorithm and RL-PPO do not perform as well.** Despite sampling over 1000 times, the greedy algorithm is constrained by previously selected passes, making it prone to getting stuck in local optima. In RL-PPO, the action mask can cause issues. When the chosen actions do not significantly impact optimization, the state value (i.e., the expected reward for the current state) may remain unchanged. Nevertheless, the action mask alters the set of selectable actions, which can affect the training of the policy network, leading to instability in the policy and fluctuations in training effectiveness.

5.2.2 Search-based experiments. In this experiment, to further demonstrate the effectiveness of our method on practical optimization goals, we shifted the target from the IR instruction count reduction, which allows for rapid evaluation, to codesize reduction (measured on the target architecture: x86_64-unknown-linux-gnu). Additionally, since the pure search methods being compared would require long search times over the complete pass space, we chose not to cluster the chained synergy pairs or train related models. Instead, we performed the search over a larger chained synergy pair

space. The chained synergy pair set used in Table 5 contains 2,700 pairs from the training set of five datasets, with codesize reduction as the optimization objective.

Table 5 compares our approach to other single-program search algorithms. Due to the expansive search space, we conducted 1,000 iterations for the random search on this synergistic graph. Regardless of whether random search or GA was applied to the chained synergy pair space, both methods significantly outperformed other algorithms such as OpenTuner, BOCA, GA, RIO, TPE, and compTuner, which operate within the 124-pass search space. Considering the search cost, we did not use samples per program due to the complexity of certain algorithms. For example, BOCA’s Bayesian optimization significantly increases the time for each sample. Additionally, the time spent on conducting iterative searches for each program in the entire test set is too long. Therefore, we randomly selected a subset of programs to test and compute the average search time. Both approaches on the chained synergy pair set also incurred minimal search costs, further demonstrating the effectiveness of the chained synergy pair search space.

5.3 Ablation Experiment

5.3.1 Connecting vs. Non-Connecting. In our method, the connection of chained synergy pairs must adhere to the rule that the pre-pass of each subsequent pair matches the post-pass of the previous pair. To assess the impact of disregarding this connectivity rule, we conducted a comparative analysis. We reselected 100 random sequences that do not adhere to the connectivity rule for each coreset to compute scores and retrain the model. After retraining, we compared the results of random strategies that also disregarded the connectivity rule with those of the previous method.

Table 6 shows that the reduction in IR instruction count achieved without following the connectivity rule is generally weaker compared to when the rule is enforced. The greatest difference in effectiveness between the two methods is observed in the NPB benchmark. As illustrated in Figure 6, adhering to the connectivity rule tends to lead to the discovery of more valuable pass sequences. This is because, without the connectivity rule, additional considerations are required regarding the influence of the pre-pass of the next pair on the post-pass of the previous pair, which reduces the advantageous scenarios for chained synergy pair connections. This demonstrates the importance of utilizing connectivity in chained synergy pairs during our search within the synergistic set.

5.3.2 Clustering vs. Non-Clustering. We evaluated the impact of clustering on the search process by obtaining the set of chained synergy pairs for each program in the test set and performing searches within this space. For programs without chained synergy pairs, we used the optimization results from Oz as a substitute. The results, as shown in Table 6,

Table 5. Comparison of codesize reduction across five datasets’ test sets for our approach versus other single-program search algorithms, with all comparisons based on the percentage reduction in codesize relative to *opt* Oz. The table also includes the average search time for each method.

Dataset	OpenTuner	BOCA	GA	RIO	TPE	compTuner	Random (Our)	GA (Our)
blas-v0	2.4%	1.1%	1.9%	2.3%	0.8%	-0.7%	4.5%	6.3%
cbench-v1	20.7%	18.6%	19.5%	19.4%	17.3%	15.1%	21.5%	22.6%
mibench-v1	7.8%	5.6%	7.6%	7.1%	5.8%	4.9%	9.1%	8.9%
npb-v0	24.0%	23.0%	23.9%	16.7%	23.3%	22.1%	25.1%	25.7%
opencv-v0	4.3%	3.7%	2.6%	4.2%	4.0%	3.6%	5.9%	5.9%
Average	11.8%	10.4%	11.1%	9.9%	10.2%	9.0%	13.2%	13.9%
Average Time (seconds)	200s	3310s	593s	200s	905s	10800s	4s	6s

Table 6. Comparison of IR instruction count reduction across different ablation scenarios, with all comparisons based on the percentage reduction relative to *opt* Oz.

Dataset	Random				GA	
	Original	Non-Connecting	Non-Clustering	Non-SL	Original	Non-Clustering
blas-v0	2.9%	2.4%	0.2%	1.8%	4.6%	0.2%
cbench-v1	4.8%	4.6%	5.1%	3.5%	5.8%	5.4%
chstone-v0	10.2%	9.9%	10.5%	8.7%	11.9%	11.2%
github-v0	1.0%	1.0%	0.1%	1.0%	1.1%	0.1%
linux-v0	0.6%	0.4%	0.03%	0.4%	0.6%	0.03%
mibench-v1	3.1%	2.2%	2.6%	2.8%	3.7%	2.7%
npb-v0	24.4%	12.1%	25.2%	14.6%	25.9%	25.5%
opencv-v0	5.4%	3.9%	3.7%	2.6%	5.9%	3.7%
poj104-v1	8.6%	8.6%	3.0%	4.7%	9.6%	3.7%
tensorflow-v0	5.8%	2.8%	1.4%	0.3%	6.1%	1.4%
Average	6.7%	4.8%	5.2%	4.0%	7.5%	5.4%

indicate that searching within the clustered search space yields significantly better outcomes compared to searching in the original search space. Moreover, the performance gap between the clustered and non-clustered search spaces widens when using more convergent search methods. This underscores the effectiveness of clustering similar programs’ chained synergy pairs together, which enhances the search results.

5.3.3 Supervised Learning vs. Non-Supervised Learning. We analyzed the score vectors of all programs in the training set to identify the chained synergy pair search space with the best average IR instruction count reduction. Using 100 random sequences from the search space, we optimized all programs in the test set to observe the effects. Table 6 shows that optimizing with the search space providing the best average performance still yields good results. This indicates that some chained synergy pairs have a degree of generality across different programs. However, the optimization results are notably weaker compared to using a unique search space selected for each program via supervised learning (SL). This underscores the uniqueness of chained synergy pairs for each program and the necessity of using specific

strategies to select more tailored chained synergy pair search spaces.

6 Conclusion

This paper presents a novel approach for finding near-optimal pass sequences for compiler optimization. Our approach leverages the synergistic relationships between compiler passes by identifying and clustering chained synergy pass pairs to create a beneficial search space. We then utilize a supervised learning model to select a suitable chained synergy pass pair set for unseen programs based on their features, effectively generalizing across different program domains. Finally, we employ diverse search strategies, including random walk, greedy algorithm, genetic algorithm and reinforcement learning, to explore the selected set and discover the near-optimal pass sequence for a given program. Our experimental results on ten benchmark datasets demonstrate that our approach achieves state-of-the-art performance in reducing IR instruction count compared to existing techniques. Notably, our approach consistently outperforms other single-program search algorithms in reducing the code-size, highlighting the effectiveness of our chained synergy

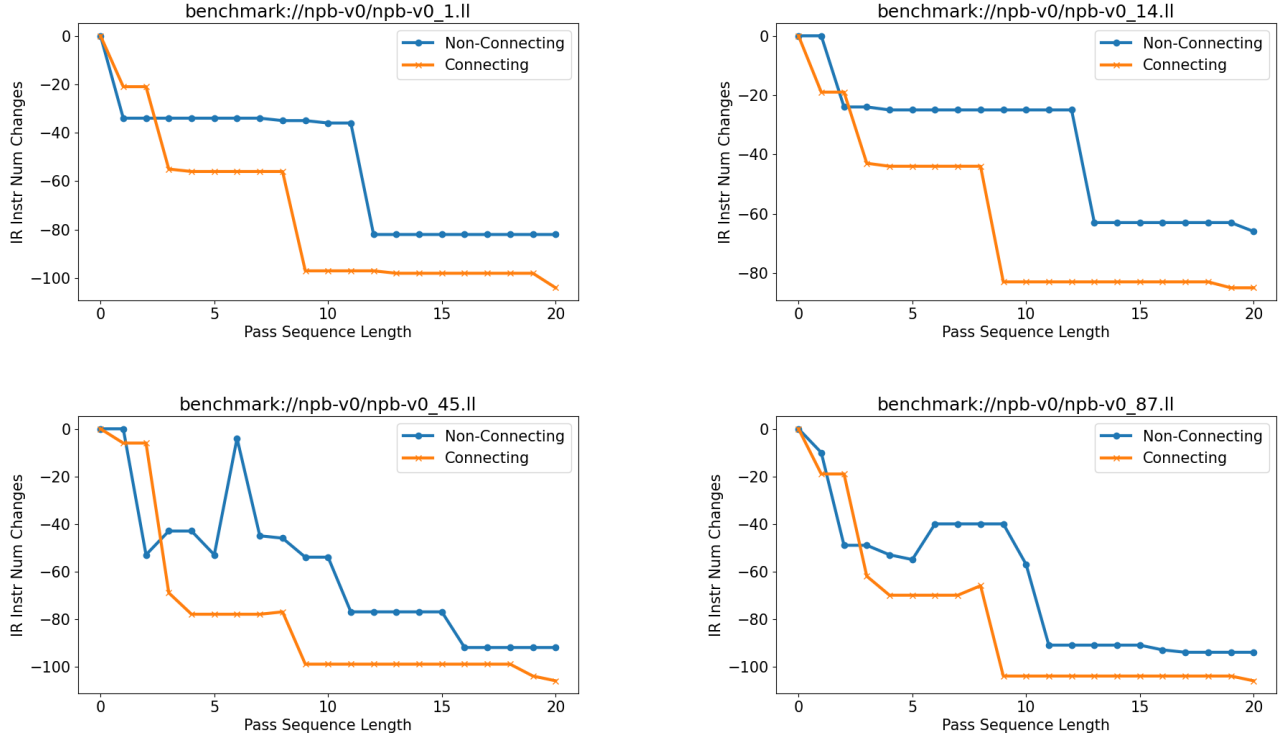


Figure 6. The cumulative IR instruction change of a program under different connection methods varies with the application of passes.

pass pairs. Furthermore, our approach is computationally efficient, significantly reducing search time compared to traditional iterative search methods.

Future work will focus on further exploring the potential of synergistic pass relationships by incorporating more sophisticated search strategies and investigating alternative clustering methods. Additionally, we plan to extend our approach to handle a wider range of optimization objectives, such as execution time and energy consumption, to enhance the practicality of our method. Furthermore, we will carefully consider the possibility that pass sequences generated using this approach may introduce program errors [13]. To mitigate this risk, we aim to design strategies that minimize the occurrence of pass sequences that could lead to incorrect program behavior or crashes, ensuring the reliability of the optimization process when combining passes.

Data-Availability Statement

This paper’s artifact is available on Zenodo [25].

Acknowledgments

This research was supported by the Strategic Priority Research Program of the Chinese Academy of Sciences, Grant No. XDA0320000 and XDA0320200.

A Artifact Appendix

A.1 Abstract

The artifacts in this paper are in the form of standalone Python scripts for the experiments. We provide artifacts, datasets, scripts as well as LLVM tools (e.g., `opt`, `clang`, `llvm-size`) to automatically generate the experimental results. Our experiments are primarily tested on LLVM 10.0.0, and different compilers and hardware may cause the results to vary. To reproduce our results, it is desirable to match hardware and software dependencies and software dependencies match. Due to the nature of the tuning methodology, each tuning run may vary. We provide docker container on google drive <https://drive.google.com/file/d/11xtUTMHRpea4uoSXg1h-F-VdAlcWrdpn/view?usp=sharing> or Aliyun mirror repository `docker pull crpi-vuemgxm9e2v73r9k.cn-hangzhou.personal.cr.aliyuncs.com/haolin564321sd/cgopaper323:latest`.

A.2 Artifact check-list

- **Algorithm:** CFSAT (Compiler Flags Synergetic Auto-tuning)
- **Program:** The datasets we selected from CompilerGym include `cbench-v1`, `mibench-v1`, `blas-v0`, `chstone-v0`, `npb-v0`, `opencv-v0`, `tensorflow-v0`, `github-v0`, `linux-v0`, and `poj104-v1`.

- **Compilation:** Does not need to be compiled, exists as a python script that can be run directly.
- **Transformations:** Target compiler will apply transformation based on the customized optimization setting.
- **Binary:** Original code is included.
- **Data set:** Datasets in CompilerGym.
- **Run-time environment:** Ubuntu22.04-LTS.
- **Hardware:** AMD EPYC 7713 64-Core Processor.
- **Metrics:** IR instruction counts and codesize.
- **Output:** Console and csv files.
- **Experiments:** Getting_Start_Guide.md in docker container.
- **How much disk space required (approximately)?:** The CFSAT code is about 2MB and the generated experimental intermediate file is about 4GB.
- **How much time is needed to prepare workflow (approximately)?:** If you want to completely reproduce the whole process it will take about 3days, if you just reproduce the main results, it will take about 6h for all scripts to be executed.
- **How much time is needed to complete experiments (approximately)?:** It takes about 6 hours to run all search strategies. It takes 30 minutes to run GA, random, or greedy searches. RL is more time-consuming.
- **Publicly available?:** Yes. Docker container <https://drive.google.com/file/d/11xtUTMHRpea4uoSXg1h-F-VdAlcWrdpn/view?usp=sharing> or Aliyun mirror repository [docker pull crpi-vuemgxm9e2v73r9k.cn-hangzhou.personal.cr.aliyuncs.com/haolin564321sd/cgopa-per323:latest](https://dockerpull.cripi-vuemgxm9e2v73r9k.cn-hangzhou.personal.cr.aliyuncs.com/haolin564321sd/cgopa-per323:latest), also available at <https://zenodo.org/records/14351920>.
- **Workflow framework used?:** Python scripts.

A.3 Description

A.3.1 How to access. We provide the Docker image through multiple methods, which can be found in the Artifact check-list.

A.3.2 Hardware dependencies. AMD EPYC 7713 64-Core Processor.

A.3.3 Software dependencies. Ubuntu 22.04, LLVM 10.0.0 and Python packages: ray, torch, numpy, ...

A.3.4 Data sets. Included in the *dataset* folder within the Docker image.

A.4 Installation

Installation is done in the provided docker image.

A.5 Experiment workflow

After downloading the Docker image and creating a container, enter the container:

```
cd /home/cfsat/
conda activate cgo_paper323
```

The following experimental scripts contain **only the experimental procedures and results of the methods** proposed

in the paper, and **do not contain the experiments and results of the other techniques compared.**

• About Table 4 main results:

– *Random Strategy:*

```
cd ./shell/
python Phase4_2_geannvp_100test.py
```

– *GA Strategy:*

```
cd ./shell/
python Phase4_2_geannvp_GAtest.py
```

– *Greedy Strategy:*

```
cd ./shell/
python Phase4_2_geannvp_Greedytest.py
```

– *RL Strategy:*

```
python run_rl.py --filepath XXX
```

XXX is the specific file path. For example, *python run_rl.py --filepath cbench-v1/cbench-v1_adpcm.ll*.

• About Table 5 main results:

– *Random Strategy:*

```
python run_codesize_Random.py \
--dataset XXX
```

– *GA Strategy:*

```
python run_codesize_GA.py --dataset XXX
```

XXX can be *blas-v0*, *cbench-v1*, *mibench-v1*, *npb-v0*, *opencv-v0*.

If the user wants to reproduce the whole process, please follow the *PhaseX_x* script inside the *shell* folder for a step-by-step reproduction.

A.6 Evaluation and expected results

For the experimental results of Table 4, the Random, Greedy, and GA strategies are saved to *./cfsat/output/* folder as *Phase4_geannvp_100Result.csv*, *Phase4_geannvp_GAResult.csv*, and *Phase4_geannvp_GreedyResult.csv*, respectively.

The RL policy is terminal output and the results of a single file run are obtained via the *episode_reward_max* output by rllib. In the experimental results in Table 5, the experimental outputs for the random and greedy strategies are also at the terminal, where *Current File* is the benchmark that is currently being tuned, and *Mean* is the total average codesize reduction percentage (compared to Oz) for all benchmarks that have been tuned up to now. The last *Mean* output is the average percentage reduction in codesize for all benchmarks in this dataset compared to Oz.

A.7 Experiment customization

How to try other compiler versions? Copy the other versions of the compiler tools (opt, clang++, llvm-size) to the *./cfsat/llvm_tools/* folder and add the action file for the corresponding version of the compiler's opt pass to *./cfsat/LLVMEnv/actionspace/*. But note that the dataset we provide is the LLVM version 10.0.0 *.ll* file. So users need to provide a compatible dataset when you change the compiler version.

How to try new benchmark? Compile the prepared benchmarks as `.ll` files with the version of the compiler that is compatible with this project, utilizing the model (`.pth`) we have provided in `./cfsat/output/` and then execute `Phase4_2_geannvp_XXXtest.py` after getting the labels.

References

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. {TensorFlow}: a system for {Large-Scale} machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 265–283.
- [2] Mohiuddin Ahmed, Raihan Seraj, and Syed Mohammed Shamsul Islam. 2020. The k-means algorithm: A comprehensive survey and performance evaluation. *Electronics* 9, 8 (2020), 1295.
- [3] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. 2014. Opentuner: An extensible framework for program autotuning. In *Proceedings of the 23rd international conference on Parallel architectures and compilation*. 303–316.
- [4] Amir H Ashouri, William Killian, John Cavazos, Gianluca Palermo, and Cristina Silvano. 2018. A survey on compiler autotuning using machine learning. *ACM Computing Surveys (CSUR)* 51, 5 (2018), 1–42.
- [5] David H Bailey, Eric Barszcz, John T Barton, David S Browning, Robert L Carter, Leonardo Dagum, Rod A Fatoohi, Paul O Frederickson, Thomas A Lasinski, Rob S Schreiber, et al. 1991. The NAS parallel benchmarks. *The International Journal of Supercomputing Applications* 5, 3 (1991), 63–73.
- [6] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for hyper-parameter optimization. *Advances in neural information processing systems* 24 (2011).
- [7] François Bodin, Toru Kisuki, Peter Knijnenburg, Mike O'Boyle, and Erven Rohou. 1998. Iterative compilation in a non-linear optimisation space. In *Workshop on profile and feedback-directed compilation*.
- [8] Shaked Brody, Uri Alon, and Eran Yahav. 2021. How attentive are graph attention networks? *arXiv preprint arXiv:2105.14491* (2021).
- [9] Junjie Chen, Ningxin Xu, Peiqi Chen, and Hongyu Zhang. 2021. Efficient compiler autotuning via bayesian optimization. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1198–1209.
- [10] Yang Chen, Shuangde Fang, Yuanjie Huang, Lieven Eeckhout, Grigori Fursin, Olivier Temam, and Chengyong Wu. 2012. Deconstructing iterative optimization. *ACM Transactions on Architecture and Code Optimization (TACO)* 9, 3 (2012), 1–30.
- [11] Ivan Culjak, David Abram, Tomislav Pribanic, Hrvoje Džapo, and Mario Cifrek. 2012. A brief introduction to OpenCV. In *2012 proceedings of the 35th international convention MIPRO*. IEEE, 1725–1730.
- [12] Chris Cummins, Zacharias V Fisches, Tal Ben-Nun, Torsten Hoefler, Michael FP O'Boyle, and Hugh Leather. 2021. Programl: A graph-based program representation for data flow analysis and compiler optimizations. In *International Conference on Machine Learning*. PMLR, 2244–2253.
- [13] Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. 2024. Meta Large Language Model Compiler: Foundation Models of Compiler Optimization. *arXiv:2407.02524 [cs.PL]* <https://arxiv.org/abs/2407.02524>
- [14] Chris Cummins, Bram Wasti, Jiadong Guo, Brandon Cui, Jason Ansel, Sahir Gomez, Somya Jain, Jia Liu, Olivier Teytaud, Benoit Steiner, et al. 2022. Compilergym: Robust, performant compiler optimization environments for ai research. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 92–105.
- [15] Grigori Fursin. 2009. Collective Tuning Initiative: automating and accelerating development and optimization of computing systems. In *GCC Developers' Summit*.
- [16] Unai Garciarena and Roberto Santana. 2016. Evolutionary optimization of compiler flag selection by learning and exploiting flags interactions. In *Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion*. 1159–1166.
- [17] Matthew R Guthaus, Jeffrey S Ringenberg, Dan Ernst, Todd M Austin, Trevor Mudge, and Richard B Brown. 2001. MiBench: A free, commercially representative embedded benchmark suite. In *Proceedings of the fourth annual IEEE international workshop on workload characterization. WWC-4 (Cat. No. 01EX538)*. IEEE, 3–14.
- [18] Ameer Haj-Ali, Qijing Jenny Huang, John Xiang, William Moses, Krste Asanovic, John Wawrzynek, and Ion Stoica. 2020. Autophase: Juggling hls phase orderings in random forests with deep reinforcement learning. *Proceedings of Machine Learning and Systems* 2 (2020), 70–81.
- [19] Yuko Hara, Hiroyuki Tomiyama, Shinya Honda, Hiroaki Takada, and Katsuya Ishii. 2008. Chstone: A benchmark program suite for practical c-based high-level synthesis. In *2008 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1192–1195.
- [20] Michael R Jantz and Prasad A Kulkarni. 2013. Exploiting phase interdependencies for faster iterative compiler optimization phase order searches. In *2013 International Conference on Compilers, Architecture and Synthesis for Embedded Systems (CASES)*. IEEE, 1–10.
- [21] Hugh Leather and Chris Cummins. 2020. Machine learning in compilers: Past, present and future. In *2020 Forum for Specification and Design Languages (FDL)*. IEEE, 1–8.
- [22] Youwei Liang, Kevin Stone, Ali Sharneli, Chris Cummins, Mostafa Elhoushi, Jiadong Guo, Benoit Steiner, Xiaomeng Yang, Pengtao Xie, Hugh James Leather, et al. 2023. Learning compiler pass orders using coresets and normalized value prediction. In *International Conference on Machine Learning*. PMLR, 20746–20762.
- [23] Hongzhi Liu, Jie Luo, Ying Li, and Zhonghai Wu. 2021. Iterative compilation optimization based on metric learning and collaborative filtering. *ACM Transactions on Architecture and Code Optimization (TACO)* 19, 1 (2021), 1–25.
- [24] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. 2016. Convolutional neural networks over tree structures for programming language processing. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 30.
- [25] Pan and Wei. 2024. *Artifact for Leveraging Synergistic Search Spaces paper at CGO2025*. <https://doi.org/10.5281/zenodo.14351920>
- [26] Sunghyun Park, Salar Latifi, Yongjun Park, Armand Behroozi, Byung-soo Jeon, and Scott Mahlke. 2022. SRTuner: Effective compiler optimization customization by exposing synergistic relations. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 118–130.
- [27] Zheng Wang and Michael O'Boyle. 2018. Machine learning in compiler optimization. *Proc. IEEE* 106, 11 (2018), 1879–1901.
- [28] Mingxuan Zhu, Dan Hao, and Junjie Chen. 2024. Compiler Autotuning through Multiple-phase Learning. *ACM Transactions on Software Engineering and Methodology* 33, 4 (2024), 1–38.

Received 2024-09-12; accepted 2024-11-04